



AI Pathfinding Simulator with Dynamic Obstacles

SUBMITTED BY: - MD FARHAN ALI

REGISTRATION NUMBER: - 25BAI10792

SUBMITTED TO: - VIT Bhopal

ACADEMIC YEAR: - 2025-2026



INTRODUCTION

Pathfinding is a fundamental problem in Artificial Intelligence with applications in robotics, navigation systems, logistics, and game development. Efficiently finding an optimal path from a start point to a goal in a dynamic environment is a critical challenge, especially when obstacles may change over time.

This project focuses on implementing and analyzing classical pathfinding algorithms such as Breadth-First Search (BFS), Uniform Cost Search (UCS), and A* within a grid-based environment. The system models both static and dynamic obstacles, allowing an agent to plan and replan its path in real time.

Additionally, the project provides a framework for comparing the performance of these algorithms based on metrics such as execution time, path cost, and nodes expanded. An interactive visualization interface is also developed to demonstrate how the agent navigates and adapts to changes in the environment.

PROBLEM STATEMENT

In real-world scenarios such as delivery systems, autonomous navigation, and robotics, agents must determine the most efficient path to a destination while dealing with obstacles and changing environments. Traditional pathfinding approaches may fail to adapt efficiently when obstacles move or when the environment changes dynamically.

The problem is to design a system that enables an agent to find an optimal path from a start location to a goal while minimizing cost and time, and to dynamically replan its path when obstacles interfere. Furthermore, the system should evaluate and compare different pathfinding algorithms to determine their efficiency and suitability for such dynamic conditions.

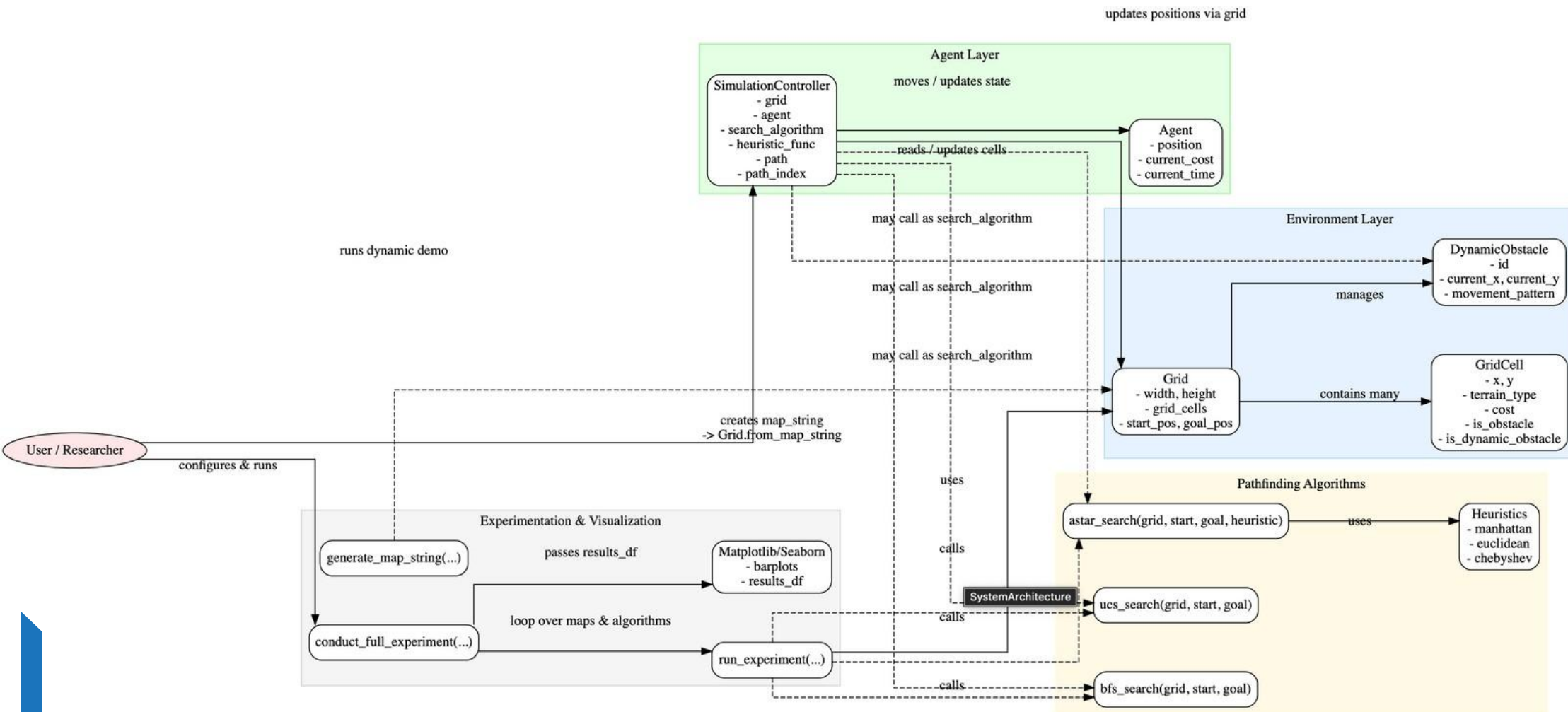
FUNCTIONAL REQUIREMENTS

- Create Maps: Generate grid-based environments with configurable size, obstacle density, and start/goal positions.
- Simulate Dynamic Obstacles: Enable obstacles to move within the grid using predefined movement patterns.
- Pathfinding Algorithms: Implement BFS, UCS, and A* algorithms to compute paths.
 - Heuristic Support: Use heuristic functions (e.g., Manhattan distance) to optimize A* search.
- Obstacle Avoidance: Ensure computed paths avoid both static and dynamic obstacles.
 - Dynamic Replanning: Recalculate paths when obstacles block the current route.
- Agent Simulation: Move an agent step-by-step along the computed path, updating position, cost, and time.
 - Performance Evaluation: Measure and compare algorithm performance using execution time, path cost, and nodes expanded.
 - Visualization: Display grid, obstacles, path, and agent movement using graphical visualization.
 - User Interface: Provide an interactive interface (Streamlit) for controlling parameters such as grid size, obstacle density, and algorithm selection.

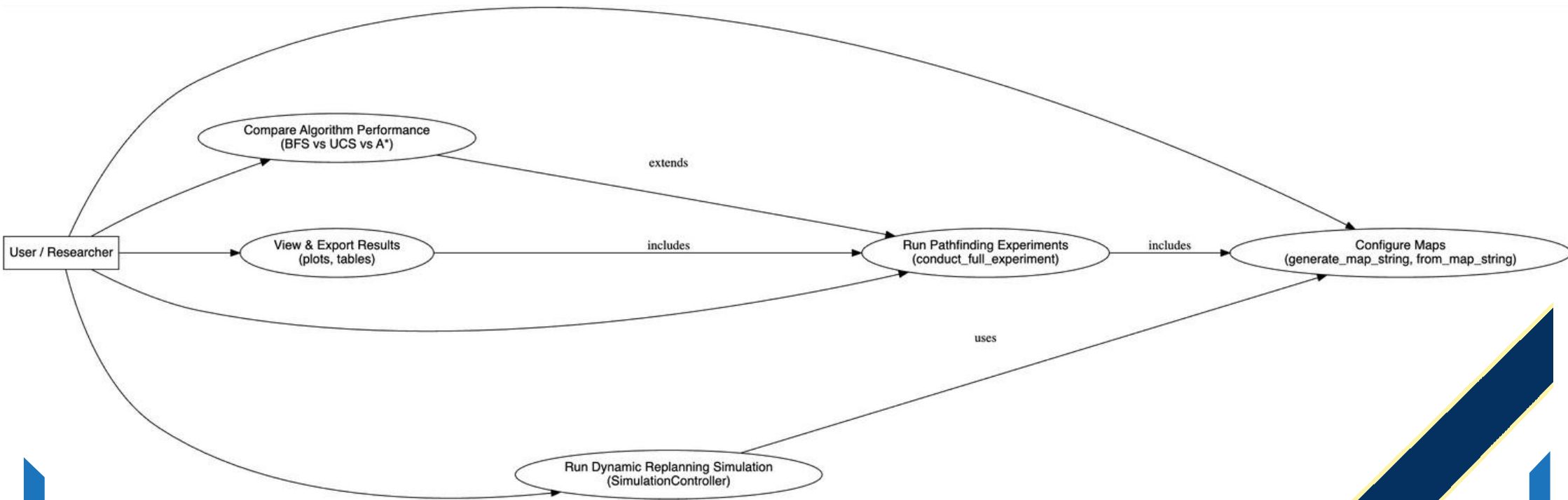
NON-FUNCTIONAL REQUIREMENTS

- **Performance:** Algorithms should compute paths within acceptable time limits even for large grids.
- **Efficiency:** Memory usage and node expansion should be minimized for optimal performance.
- **Scalability:** The system should handle increasing grid sizes and obstacle densities without significant performance degradation.
- **Usability:** The interface should be intuitive, allowing users to easily configure and run simulations.
- **Reliability:** The system should consistently produce correct paths under both static and dynamic conditions.
- **Maintainability:** Code should be modular, readable, and easy to extend with new algorithms or features.
- **Responsiveness:** Visualization and UI updates should occur smoothly without noticeable delay.
- **Portability:** The system should run on standard environments without requiring specialized hardware.

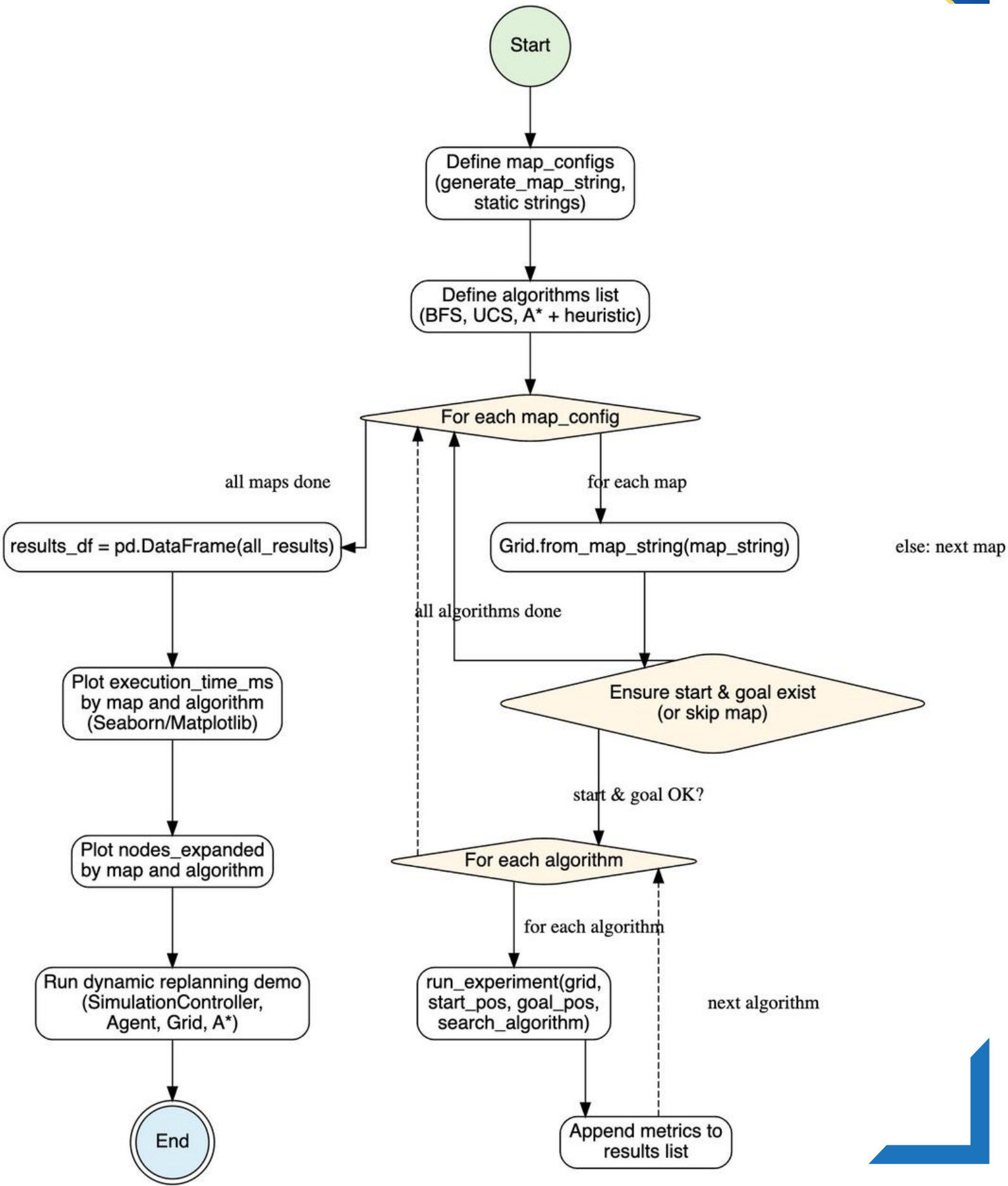
SYSTEM ARCHITECTURE



USE CASE DIAGRAM



WORKFLOW DIAGRAM



SEQUENCE DIAGRAM



DESIGN DECISIONS & RATIONALE

- **Modular Design:** The system is divided into components such as Grid, Agent, and SimulationController to ensure better organization, readability, and maintainability.
- **Grid-Based Representation:** A 2D grid is used to model the environment due to its simplicity and ease of implementing neighbor-based movement and obstacle handling.
- **Multiple Algorithms:** BFS, UCS, and A* are implemented to compare different search strategies and analyze their performance under varying conditions.
- **Heuristic Optimization:** A* uses the Manhattan distance heuristic, which is well-suited for grid environments and significantly improves search efficiency compared to uninformed methods.
- **Dynamic Replanning:** The system is designed to replan paths when dynamic obstacles interfere, enabling the agent to adapt to changing environments in real time.
- **Data Structures:** Queues are used for BFS, while priority queues (heaps) are used for UCS and A* to efficiently select the next node based on cost or heuristic.
- **Performance Evaluation:** Metrics such as execution time, nodes expanded, and path cost are tracked to provide a quantitative comparison of algorithms.
- **Visualization & UI:** A Streamlit-based interface is used to provide an interactive and intuitive way to visualize the grid, agent movement, and algorithm behavior.
- **Trade-offs Consideration:** BFS guarantees shortest paths but is inefficient for large maps, UCS handles weighted paths but is slower, while A* provides an optimal balance between efficiency and accuracy using heuristics.

IMPLEMENTATION

DETAILS

- **Map Generation:** Grid maps are generated using a function that creates a 2D environment based on specified width, height, and obstacle density. Start and goal positions are either predefined or randomly assigned.
- **Grid Representation:** The map string is parsed into a Grid object, where each cell represents a location with properties such as obstacle presence and traversal cost.
- **Agent Implementation:** An Agent class is used to track the agent's position, path cost, and time steps. The agent moves step-by-step along the computed path.
- **Pathfinding Algorithms:**
 - BFS uses a queue to explore nodes level by level.
 - UCS uses a priority queue to expand nodes with the lowest cumulative cost.
 - A* uses a priority queue combined with a heuristic (Manhattan distance) to efficiently find the optimal path.
- **Simulation Controller:** A Simulation Controller manages the interaction between the agent and the grid. It executes path planning, movement, and environment updates.
- **Dynamic Obstacles:** Obstacles can move based on predefined movement patterns. Their positions are updated at each simulation step.
- **Replanning Mechanism:** If a dynamic obstacle blocks the agent's current path, the system triggers a replanning process using the selected algorithm to compute a new optimal path.
- **Performance Tracking:** The system records execution time, number of nodes expanded, and path cost for each algorithm to enable comparison.
- **Visualization:** Matplotlib is used to display the grid, path, obstacles, and agent movement.
- **User Interface:** A Streamlit-based interface allows users to configure parameters such as grid size, obstacle density, and algorithm selection, and visualize the simulation.

SCREENSHOTS AND RESULTS

```
> __pycache__
v .vscode
  {} settings.json
> images
  agent_pathfinding_en...
  app.py
  main.py
  i readme.md
  statement.md
  visualizer.py
```

```
--- Setting Up Maps ---

Small Map:
S....
.....
##...
.....
....G

Medium Map Loaded

Large Map Loaded

Dynamic Map:

S.X
.D.
.G.
```

```
--- Running Dynamic Replanning Simulation (Proof-of-Concept) ---
Initial path planned: [(0, 0), (0, 1), (0, 2), (1, 2)] with cost 3.00

>>> Simulation Step 1 <<<

--- Updating Dynamic Obstacles ---
Obstacle 0 moved to (0, 1)
!!! Collision detected at next step (0, 1)! Replanning needed.
--- Replanning Path ---
New path planned: [(0, 0), (1, 0), (1, 1), (1, 2)] with cost 3.00
Agent moved to (1, 0). Total cost: 1.00, Time: 1
Agent Status: Position=(1, 0), Cost=1.00, Time=1

>>> Simulation Step 2 <<<

--- Updating Dynamic Obstacles ---
Obstacle 0 moved to (0, 1)
Agent moved to (1, 1). Total cost: 2.00, Time: 2
Agent Status: Position=(1, 1), Cost=2.00, Time=2

>>> Simulation Step 3 <<<

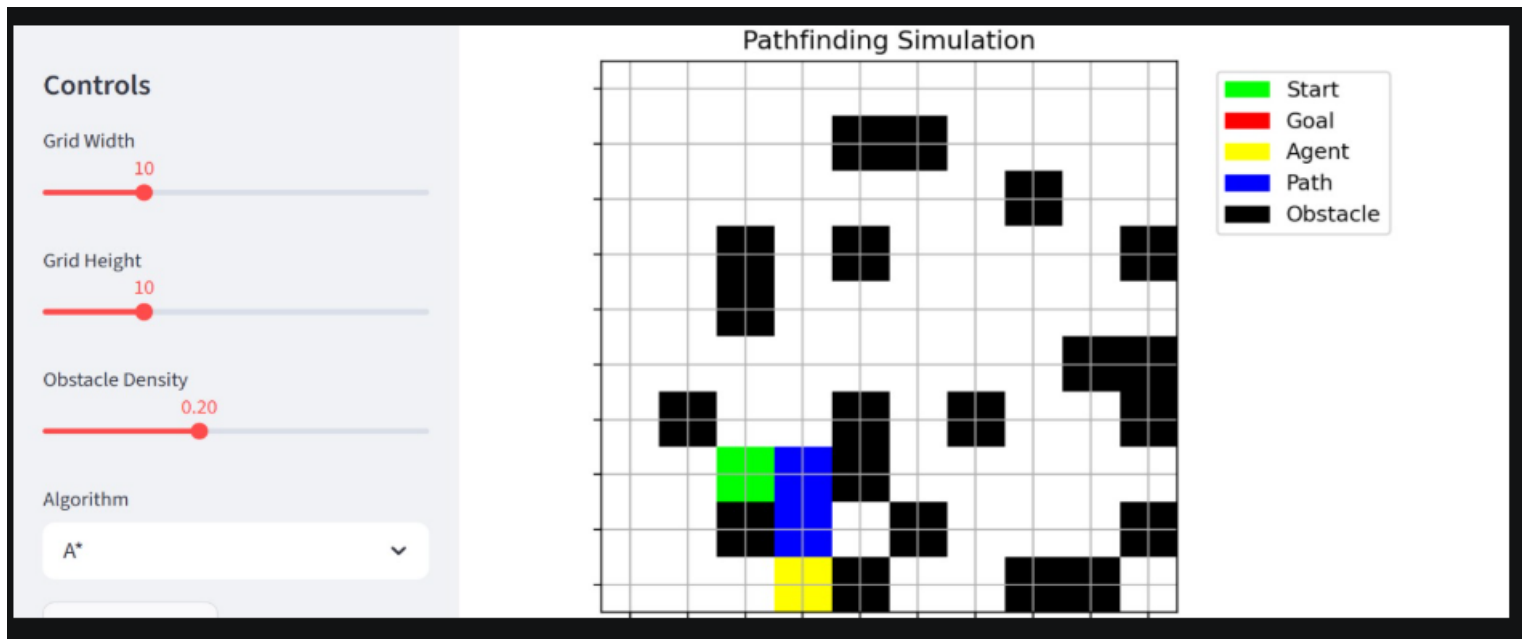
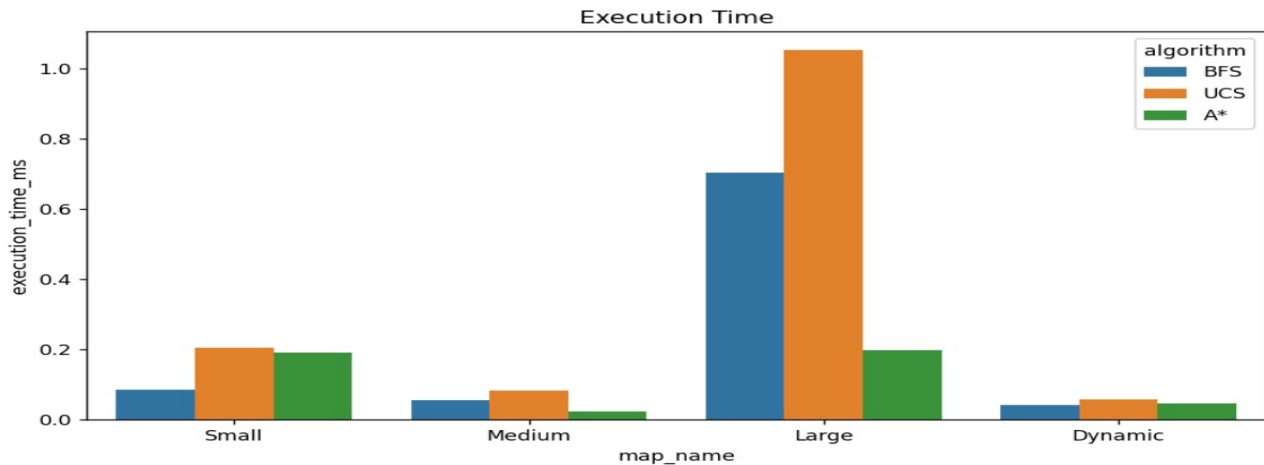
--- Updating Dynamic Obstacles ---
Obstacle 0 moved to (0, 1)
Agent moved to (1, 2). Total cost: 3.00, Time: 3
Agent reached the goal!
Agent Status: Position=(1, 2), Cost=3.00, Time=3

--- Dynamic Replanning Simulation Summary ---
Final Agent Position: (1, 2)
Total Cost: 3.00
Total Time Steps: 3
Goal Reached: True

All simulations and experiments completed.
```

--- RESULTS ---											
	algorithm	start_pos	goal_pos	path_found	path_length	path_cost	execution_time_ms	nodes_expanded	map_name	map_width	map_height
0	BFS	(0, 0)	(4, 4)	True	9	8	0.1691	22	Small	5	5
1	UCS	(0, 0)	(4, 4)	True	9	8	0.1605	23	Small	5	5
2	A*	(0, 0)	(4, 4)	True	9	8	0.1765	19	Small	5	5
3	BFS	(2, 8)	(5, 14)	True	14	13	0.4995	106	Medium	15	15
4	UCS	(2, 8)	(5, 14)	True	14	13	0.6987	109	Medium	15	15
5	A*	(2, 8)	(5, 14)	True	14	13	0.2128	29	Medium	15	15
6	BFS	(12, 9)	(15, 11)	True	12	11	0.5028	138	Large	25	25
7	UCS	(12, 9)	(15, 11)	True	12	11	0.9782	146	Large	25	25
8	A*	(12, 9)	(15, 11)	True	12	11	0.2578	34	Large	25	25
9	BFS	(0, 0)	(1, 2)	True	4	3	0.0839	6	Dynamic	3	3
10	UCS	(0, 0)	(1, 2)	True	4	3	0.0485	5	Dynamic	3	3
11	A*	(0, 0)	(1, 2)	True	4	3	0.0455	5	Dynamic	3	3

SCREENSHOTS AND RESULTS



TEST

APPROACH

- Experiment Setup: Multiple grid environments of varying sizes and obstacle densities are generated to evaluate algorithm performance under different conditions.
- Algorithm Execution: Each pathfinding algorithm (BFS, UCS, and A*) is applied to the same set of maps to ensure a fair comparison.
- Performance Measurement: Key metrics such as execution time, path cost, and number of nodes expanded are recorded for each algorithm.
- Comparative Analysis: The collected results are analyzed to determine the efficiency, optimality, and scalability of each algorithm.
- Dynamic Scenario Testing: Special test cases with moving obstacles are used to evaluate the system's ability to replan paths in real time.
- Visualization: Graphs and grid-based visualizations are used to clearly demonstrate algorithm behavior and performance differences.

CHALLENGES FACED

- Handling Dynamic Obstacles: Implementing moving obstacles and ensuring that the agent correctly detects changes and replans paths in real time was a major challenge.
- Algorithm Correctness: Ensuring that BFS, UCS, and A* produced correct and optimal paths required careful implementation and debugging.
 - Performance Optimization: Managing execution time and memory usage, especially for larger grids, was challenging due to increased node expansions.
 - Environment Setup: Designing flexible grid generation with varying obstacle densities and ensuring proper placement of start and goal positions required careful handling.
 - Integration Issues: Connecting different modules such as map generation, pathfinding algorithms, and simulation controller required proper coordination.
 - Import and Module Management: Handling file imports and ensuring correct linking between main execution files and environment modules posed initial difficulties.
- Visualization Synchronization: Ensuring that the visualization accurately reflected real-time simulation steps and agent movement required careful coordination between logic and display.

LEARNING AND KEY TAKEAWAYS

- **Understanding of Pathfinding Algorithms:** Gained practical knowledge of BFS, UCS, and A* algorithms and how they differ in terms of efficiency and optimality.
- **Heuristic-Based Optimization:** Learned how heuristic functions such as Manhattan distance improve the performance of A* search.
- **Handling Dynamic Environments:** Developed the ability to design systems that adapt to changes, such as moving obstacles, through real-time path replanning.
- **Performance Analysis:** Gained experience in evaluating algorithms using metrics like execution time, nodes expanded, and path cost.
- **System Design Skills:** Improved understanding of modular design by separating components such as Grid, Agent, and SimulationController.
- **Visualization and UI Development:** Learned how to represent algorithm behavior visually using Matplotlib and build interactive interfaces using Streamlit.
- **Problem-Solving Skills:** Strengthened the ability to debug complex systems involving multiple interacting components.
- **Real-World Application Understanding:** Understood how pathfinding techniques are applied in domains like navigation systems, robotics, and logistics.

FUTURE ENHANCEMENTS

- **Additional Algorithms:** Extend the system by implementing advanced search methods such as Greedy Best-First Search, Weighted A*, and Dijkstra's algorithm for deeper comparison.
- **Intelligent Obstacles:** Introduce more complex obstacle behaviors using probabilistic or rule-based movement patterns to simulate realistic environments.
- **Enhanced Visualization:** Improve the UI with smoother animations, color-coded legends, and real-time updates for better understanding of agent behavior.
- **Machine Learning Integration:** Incorporate learning-based approaches such as Reinforcement Learning (e.g., Deep Q-Networks) to enable the agent to learn optimal paths over time.
- **Large-Scale Environments:** Optimize the system to handle larger grids and more complex terrains efficiently.
- **Multi-Agent Systems:** Extend the model to support multiple agents navigating simultaneously while avoiding collisions.
- **Real-World Integration:** Adapt the system for real-world applications such as traffic routing, warehouse automation, and robotics navigation.
- **Performance Optimization:** Improve computational efficiency by reducing node expansions and optimizing data structures.

REFERENCES

- Stuart Russell and Peter Norvig, "Artificial Intelligence: A Modern Approach", Pearson.
- Python Official Documentation: <https://docs.python.org/>
- Pandas Documentation: <https://pandas.pydata.org/docs/>
- Matplotlib Documentation: <https://matplotlib.org/stable/>
- Streamlit Documentation: <https://docs.streamlit.io/>
- GitHub: <https://github.com/>

THANK
YOU